



FACULTAD DE INGENIERÍA Y CIENCIAS APLICADAS

MAESTRÍA EN INTELIGENCIA ARTIFICIAL APLICADA

PROYECTO MIA

PROPUESTA DEL PROYECTO DE TITULACIÓN

Modelo predictivo del rendimiento deportivo en el fútbol ecuatoriano a partir del presupuesto institucional y variables competitivas usando técnicas de Inteligencia Artificial

TUTOR: Medina Sotomayor Jaime Felipe.

AUTOR1: Castro García Diego Fernando

AUTOR2: Pila Caiza Cesar Silvio

Índice

1. Prototipo final - implementación y completitud
 - 1.1 Descripción del problema y solución
 - 1.2 Arquitectura del prototipo
 - 1.3 Modelo final entrenado y validado
 - 1.4 Mejoras incorporadas a lo largo del curso
 - 1.5 Evidencia de pruebas finales
2. Reproducibilidad y evidencias
 - 2.1 Requisitos técnicos y dependencias
 - 2.2 Instrucciones de ejecución paso a paso
 - 2.3 Pipeline general (datos -> modelo -> inferencia)
 - 2.4 Control de versiones
3. Organización estructurada del repositorio
 - 3.1 Mapa de carpetas
 - 3.2 Resultados finales y métricas comparativas
 - 3.3 Evidencias de robustez (referencia a anexos S9)
4. Proyecto
 - 4.1 Contexto y problema abordado
 - 4.2 Enfoque metodológico
 - 4.3 Resultados y hallazgos
 - 4.4 Consideraciones éticas, riesgos y limitaciones
5. Reflexión final y uso de la retroalimentación

1. Prototipo final - implementación y completitud

1.1 Descripción del problema y solución

Los clubes de la LigaPro Serie A planifican cada temporada con presupuestos muy heterogéneos (rango 4 - 25 MUSD) y una alta incertidumbre sobre el rendimiento deportivo esperado. La directiva necesita una herramienta cuantitativa, reproducible y explicable que estime los puntos finales de la temporada antes del primer partido, para apoyar la negociación con patrocinadores, la fijación de objetivos deportivos y la rendición de cuentas ante el directorio.

La solución entregada es un modelo predictivo entrenado con datos reales de 16 clubes a lo largo de 8 temporadas (2019-2026, 128 filas) que combina variables presupuestarias (presupuesto, salarios, inversión en refuerzos) con variables competitivas pre-temporada (plantilla, cuerpo técnico, contexto, rendimiento rezagado de la temporada anterior). El modelo se expone a través de una interfaz web.

1.2 Arquitectura del prototipo

El prototipo se organiza en tres capas claramente desacopladas que permiten reentrenar, depurar y desplegar de manera independiente:

- Capa de datos. Fuentes apropiadas en CSV organizadas por categoría (presupuestarias / competitivas) y dataset (ver scripts/consolidar_dataset.py).
- Capa de modelado. Tres pipelines comparados (Ridge, Random Forest, XGBoost) entrenados sobre la misma matriz de features pre-temporada y evaluados sobre splits temporales (train 2019-2023, val 2024, test 2025). El mejor por MAE en validación se promueve a mejor_modelo.joblib (ver scripts/entrenamiento.py).
- Capa de inferencia y UI. Página web estática (output/index.html) que carga los modelos exportados a JSON (ridge_model.json, rf_model.json, xgb_model.json) y ejecuta la predicción en JavaScript. No requiere backend; el archivo se abre directamente en el navegador o se sirve con ``python -m http.server``.

Tabla. Componentes del prototipo final

Componente	Archivo / carpeta	Función
Datos	data/categorías/	CSV por categoría (presupuestos, plantilla, posiciones, etc.)
Dataset modelable	data/datasets/dataset_final.csv	128 filas x 24 cols con features y target.
Consolidación	scripts/consolidar_dataset.py	Une categorías y agrega variables rezagadas.
Preprocesamiento	scripts/preprocesamiento.py	Define FEATURES, TARGET y splits temporales.
Entrenamiento	scripts/entrenamiento.py	Entrena 3 modelos y selecciona el mejor por MAE val.
Exportación	scripts/exportar_modelos_json.py	Convierte. joblib -> .json para uso en navegador.
Modelos	output/models/*.joblib + *.json	Artefactos entrenados (Python y JavaScript).
Card del modelo	output/models/model_card.json	Métricas, features, partición y mejor modelo.
Interfaz web	output/index.html	UI offline con predicción en JavaScript.
Inferencia CLI	scripts/predecir_equipo.py	Predicción interactiva por terminal.
Notebook EDA	notebooks/01_ligapro_COLAB.ipynb	Exploración y modelado en Colab.

Nota. Esta tabla resume los componentes que conforman el prototipo final. Cada componente es ejecutable de manera independiente; el archivo model_card.json actúa como contrato de interfaz entre la capa de modelado y la capa de inferencia.

1.3 Modelo final entrenado y validado

El modelo final seleccionado por el script de entrenamiento, con base en el menor MAE en el conjunto de validación (temporada 2024), es RIDGE. Esta elección se documenta en output/models/model_card.json y se materializa en el archivo mejor_modelo.joblib.

El conjunto de variables predictoras se restringe deliberadamente a información conocida ANTES de que comience la temporada, lo que garantiza la validez del modelo como herramienta de planificación pre-temporada y elimina cualquier riesgo de fuga temporal.

Tabla. Variables (features) usadas por el modelo final

Bloque	Variable	Tipo
Recursos	presupuesto_musd	numerica
Recursos	salario_promedio_kusd	numerica
Recursos	inversion_refuerzos_musd	numerica
Plantilla	edad_promedio_plantilla	numerica
Plantilla	extranjeros_plantilla	entera
Cuerpo tecnico	antiguedad_dt_meses	entera
Contexto	participacion_internacional	entera (partidos)
Contexto	capacidad_estadio	entera
Inercia	posicion_temporada_anterior	ordinal
Lag deportivo	goles_favor_ant	entera
Lag deportivo	goles_contra_ant	entera
Lag deportivo	diferencia_goles_ant	entera
Lag deportivo	xg_ant	numerica
Lag deportivo	posesion_ant	% (0-100)
TARGET	puntos_temporada	numerica (0-90)

Nota. Las variables Lag deportivo corresponden a estadísticas de la temporada anterior por equipo (shift de 1 periodo). Para la primera temporada (2019) se imputa la mediana del bloque, evitando NaN sin introducir información futura.

1.4 Mejoras incorporadas a lo largo del curso

El prototipo evoluciono semana a semana incorporando las retroalimentaciones recibidas. Las principales mejoras consolidadas en esta entrega final son:

- S2 - S3. Reorganización del dataset por categoría (presupuestaria / competitiva) en data/categorías/, con un script de consolidación (consolidar_dataset.py) que produce el dataset modelable. Esto mejora la trazabilidad y la mantenibilidad de las fuentes.
- S4 - S5. Reformulación del problema de regresión: se descartó la predicción directa de posición (ordinal con empates) y se adoptó la predicción de puntos_temporada, más estable y con mejor poder predictivo.
- S6 - S7. Comparación sistemática de tres modelos (Ridge, Random Forest, XGBoost) con métricas robustas (MAE, RMSE, R2 y Spearman). Se identificó el sobreajuste de los modelos no lineales con 80 filas de entrenamiento y se justificó la elección de Ridge.
- S8. Construcción de la interfaz web (output/index.html) con diseño responsive y selector de modelo, sustituyendo el primer prototipo de línea de comandos.
- S9. Anexos de validación (docs/S9-ANEXOS DE VALIDACION.docx) con evidencia cuantitativa de validación, robustez e interpretabilidad.
- S10. Eliminación de la dependencia del backend FastAPI mediante la exportación de los modelos a JSON (scripts/exportar_modelos_json.py) y la reimplementación de la inferencia en JavaScript.

1.5 Evidencia de pruebas finales

Se realizaron las siguientes pruebas de aceptación sobre la versión final entregada:

Tabla. Pruebas finales realizadas sobre el prototipo

#	Prueba	Resultado
P1	Reentrenamiento end-to-end (consolidar -> entrenar)	OK - regenera dataset_final.csv y los 3 modelos sin error.
P2	Carga de modelos desde JSON en JavaScript	OK - los pesos Ridge / RF / XGB se aplican.
P3	Apertura de output/index.html en navegador	OK - UI carga, formularios validan, predicción responde.
P4	Predicción CLI con scripts/predecir_equipo.py	OK - reproduce el mismo valor que la UI.

P5	Consistencia val 2024 entre CLI y UI	OK - diferencia < 1e-2 puntos.
P6	Partición temporal sin fuga	OK - val=2024, test=2025 nunca aparecen en train.
P7	Manejo de la primera temporada (2019)	OK - lags imputados con la mediana del bloque.

Nota. Las pruebas P1-P5 cubren el ciclo completo desde los CSV crudos hasta la respuesta visible al usuario. P6-P7 son pruebas de rectitud metodológica que blindan el modelo contra fugas y NaN.

2. Reproducibilidad y evidencias

2.1 Requisitos técnicos y dependencias

El prototipo se ejecuta sobre Python 3.10 o superior y un navegador moderno (Chrome, Firefox, Edge). Las dependencias del backend de entrenamiento se declaran en output/requirements.txt:

- Fastapi
- Uvicorn
- Joblib
- Pandas
- scikit-learn
- xgboost
- pydantic

Para la versión online en navegador (entrega final) no es necesario instalar dependencias: basta con abrir output/index.html. Las librerías FastAPI / Uvicorn solo se requieren si se desea levantar el backend opcional descrito en output/main.py.

2.2 Instrucciones de ejecución paso a paso

Reproducir el prototipo desde cero requiere los siguientes pasos:

Tabla. Pasos para reproducir el prototipo

#	Comando / acción	Salida esperada
1	git clone <repo> && cd Modelo-Predictivo-Liga-Pro-Seria-A	Repositorio local.
2	python -m venv .venv && activate	Entorno virtual aislado.
3	pip install -r output/requirements.txt	Dependencias instaladas.
4	python scripts/consolidar_dataset.py	data/datasets/dataset_final.csv (128 filas).

5	python scripts/entrenamiento.py	output/models/*.joblib + model_card.json.
6	python scripts/exportar_modelos_json.py	output/models/*_model.json.
7	Abrir output/index.html en el navegador	UI lista para inferencia offline.
8	Opcional: python -m http.server 5500 dentro de output/	http://localhost:5500

Nota. Los pasos 4-6 son el pipeline reproducible de modelado; los pasos 7-8 corresponden al uso final del producto. La separación permite reentrenar sin tocar la UI y viceversa.

2.3 Pipeline general

El pipeline completo del proyecto se ilustra a continuación:

data/categorias/*.csv

scripts/consolidar_dataset.py --> data/datasets/dataset_final.csv

scripts/preprocesamiento.py --> FEATURES, TARGET, train/val/test

scripts/entrenamiento.py --> Ridge / RandomForest / XGBoost

output/models/*.joblib --> output/models/*_model.json

scripts/exportar_modelos_json.py --> UI web

2.4 Control de versiones

El proyecto se versiona con Git en el repositorio publico github.com/parametrico005/modelo-predictivo-liga-pro-seria-a. La rama principal (main) contiene la version estable.

Los artefactos binarios críticos (modelos .joblib y .json, dataset consolidado) se versionan en el mismo repositorio para garantizar que cualquier evaluador pueda reproducir la inferencia sin necesidad de reentrenar.

3. Organización estructurada del repositorio

3.1 Mapa de carpetas

Estructura del repositorio

```
LigaPro Serie A/  
├── README.md  
├── cover.pdf          <- portada institucional  
├── data/  
│   ├── categorias/   <- FUENTE CANONICA (CSV)  
│   │   ├── presupuestarias/  
│   │   │   └── ligapro_presupuestos.csv    (presupuesto, salario,  
│   │   │                                       refuerzos, valor plantel,  
│   │   │                                       cantidad fichajes, etc.)  
│   │   ├── competitivas/  
│   │   │   ├── ligapro Equipos.csv         (master de clubes)  
│   │   │   ├── ligapro plantilla.csv       (edad / extranjeros)  
│   │   │   ├── ligapro cuerpo tecnico.csv  (DT)  
│   │   │   ├── ligapro participacion_internacional.csv  
│   │   │   ├── ligapro rendimiento_ofensivo.csv  
│   │   │   ├── ligapro rendimiento_defensivo.csv  
│   │   │   ├── ligapro posiciones.csv  
│   │   │   └── ligapro eficiencia.csv      (puntos / posibles)  
│   └── datasets/     <- INPUT DEL MODELO  
│       ├── dataset_final.csv    (128 filas, 24 cols)  
│       ├── train.csv           (2019-2023)  
│       ├── val.csv             (2024)  
│       └── test.csv            (2025)  
├── scripts/  
│   ├── consolidar_dataset.py    (categorias -> dataset_final)  
│   ├── preprocesamiento.py      (FEATURES, TARGET, splits)  
│   ├── entrenamiento.py         (entrena los 3 modelos)  
│   ├── exportar_modelos_json.py (.joblib -> .json para web)  
│   ├── prediccion.py           (inferencia batch desde CSV)  
│   ├── predecir_equipo.py       (inferencia interactiva)  
│   ├── generar_s9_anexos.py     (genera S9-ANEXOS.docx)  
│   ├── generar_s10_informe_final.py (genera S10-INFORME.docx)  
│   └── generar_s10_pdf.py       (genera S10-INFORME.pdf)  
├── output/  
│   ├── index.html              <- UI + MODELOS  
│   ├── main.py                 (UI moderna offline)  
│   ├── requirements.txt  
│   └── models/  
│       ├── ridge.joblib  
│       ├── randomforest.joblib  
│       ├── xgboost.joblib  
│       ├── mejor_modelo.joblib   (copia del mejor por MAE)  
│       ├── ridge_model.json      (pesos para inferencia JS)  
│       ├── rf_model.json  
│       ├── xgb_model.json  
│       └── model_card.json       (metricas + features + splits)  
└── notebooks/  
    └── 01_ligapro_COLAB.ipynb   (EDA + modelado)
```

3.2 Resultados finales y métricas comparativas

Los tres modelos se entrenan con los mismos features sobre el split train (2019-2023, 80 filas) y se evalúan en val (2024, 16 filas) y test (2025, 16 filas). El mejor por MAE en validación es Ridge.

Tabla. Métricas comparativas en validación (temporada 2024)

Modelo	MAE	RMSE	R2	Spearman
Ridge (mejor)	6.6512	7.828	0.6264	0.7237
Random Forest	9.2815	10.9369	0.2708	0.4657
XGBoost	11.388	13.25	-0.0703	0.2741

Nota. Ridge gana en validación porque con 80 filas de entrenamiento los modelos no lineales sobreajuste. La métrica MAE = 6.65 puntos implica un error típico inferior a 2 victorias en una temporada de 30 partidos, lo que es competitivo para un horizonte pre-temporada.

Tabla. Métricas comparativas en test (temporada 2025)

Modelo	MAE	RMSE	R2	Spearman
Ridge	14.7815	17.1671	-0.195	0.5771
Random Forest	12.9719	15.6052	0.0126	0.6273
XGBoost	12.9562	15.4101	0.0371	0.5063

Nota. El R2 negativo en Ridge-test refleja que la temporada 2025 tuvo una varianza atípica (cambio normativo en cupos de extranjeros y nuevos ascendidos). La correlación de Spearman se mantiene > 0.5 en los tres modelos, lo que confirma que el ranking se predice de forma razonable aunque las puntuaciones absolutas se desvíen.

3.3 Evidencias de robustez

Las evidencias detalladas de robustez están documentadas en el anexo técnico docs/S9-ANEXOS DE VALIDACION.docx, que incluye:

4. Proyecto

4.1 Contexto y problema abordado

Apertura. La LigaPro Serie A es la máxima categoría del fútbol ecuatoriano: 16 clubes, 30 partidos por temporada, presupuestos que varían de 4 a 25 millones de dólares. Esta heterogeneidad presupuestaria convive con una alta incertidumbre deportiva: cada año, en promedio, 5 de los 16 clubes pelean el descenso y otros 5 pelean cupo a Copas internacionales.

Justificación. Las directivas de los clubes necesitan estimar de manera defendible cuantos puntos pueden lograr ANTES de iniciar la temporada, para fijar objetivos realistas, negociar patrocinios y explicar al directorio el retorno esperado de la inversión en la plantilla. Hoy esa estimación se hace de forma intuitiva. Nuestro proyecto la convierte en un modelo cuantitativo, reproducible y explicable.

Pregunta de investigación. ¿Es posible predecir, antes del primer partido, los puntos finales de cada club de la LigaPro a partir de su presupuesto y un conjunto reducido de variables competitivas?

4.2 Enfoque metodológico

Tipos de datos utilizados:

- Presupuestarios. Presupuesto anual (MUSD), salario promedio, inversión en refuerzos, valor de plantel y cantidad de fichajes. Fuentes: Transfermarkt, prensa especializada, CONMEBOL.
- Plantilla y cuerpo técnico. Edad promedio, número de extranjeros y antigüedad del DT en meses al inicio de temporada.
- Contexto. Capacidad del estadio y participación internacional (número de partidos clasificados a Libertadores / Sudamericana).
- Rendimiento rezagado. Goles a favor / en contra, diferencia, xG y posesión de la temporada anterior (lags), para capturar inercia deportiva sin introducir fuga temporal.

Modelos aplicados. Se compararon tres modelos de regresión sobre el mismo target (puntos_temporada): (i) Ridge como línea base interpretable; (ii) Random Forest con 500

árboles para capturar no linealidades; (iii) XGBoost con 400 estimadores para boosting secuencial.

Principales decisiones técnicas:

- Partición temporal estricta: train 2019-2023, val 2024, test 2025 (sin fuga).
- Imputación de lags por la mediana SOLO sobre la primera temporada (2019).
- Selección automática del mejor modelo por MAE en validación.
- Inferencia 100% offline en navegador vía exportación .joblib -> .json.

4.3 Resultados y hallazgos

Métricas finales alcanzadas. El mejor modelo es RIDGE con MAE = 6.6512 puntos y Spearman = 0.7237 en validación. Esto significa que, en promedio, el modelo se equivoca por menos de 7 puntos (poco más de dos victorias) y predice correctamente el orden de la tabla con una correlación fuerte (0.72).

Interpretación. Ridge gana porque con 80 filas de entrenamiento los modelos no lineales (RF, XGB) sobre ajustan: tienen MAE val de 9.3 y 11.4 respectivamente. Esto valida que en datasets pequeños y de señal lineal, la regularización L2 es competitiva con métodos más complejos. Los coeficientes estandarizados de Ridge muestran que el presupuesto y la inversión en refuerzos son los predictores más fuertes, seguidos por la inercia (puntos / posición del año previo).

Impacto potencial. Aplicado al inicio de cada temporada, el modelo permite a la directiva: (a) construir un objetivo numérico defendible (rango de puntos esperado +- MAE), (b) comparar el plan deportivo frente a clubes pares con presupuesto similar y (c) cuantificar el impacto marginal de invertir 1 MUSD adicional en refuerzos.

4.4 Consideraciones éticas, riesgos y limitaciones

Principales riesgos identificados:

- Sesgo de muestra. Solo 8 temporadas y 16 clubes; los resultados no son extrapolables a otras ligas sin re-entrenamiento.

- Determinismo aparente. Mostrar el modelo predice 65 puntos puede ser leído como un veredicto. Mitigación: la UI muestra siempre el MAE como banda de incertidumbre.
- Riesgo reputacional para clubes. Una predicción publica baja podría afectar la moral o la negociación de patrocinios. Mitigación: el prototipo se entrega como herramienta interna de planificación, no como pronóstico público.
- Cambios normativos. Modificaciones a los cupos de extranjeros, formato de campeonato o calendario rompen los supuestos. Mitigación: el reentrenamiento está automatizado en un solo comando.

Estrategias de mitigación aplicadas:

- Reporte siempre acompañado de MAE y banda Spearman.
- Partición temporal estricta para evitar leakage del futuro.
- Comparación frente a baseline lineal interpretable (Ridge).
- Anexos S9 con pruebas de robustez y sensibilidad documentadas.

Limitaciones del modelo:

- Los R^2 sobre test 2025 son negativos en Ridge: la temporada 2025 fue atípica por la incorporación de dos clubes recién ascendidos con presupuestos reportados parcialmente. La correlación de ranking sigue siendo positiva (Spearman = 0.58).
- El modelo no captura eventos fuera del dataset entrenado (lesiones, suspensiones, cambios de DT a mitad de temporada), por diseño: solo opera con información pre-temporada.
- El target son puntos de fase regular, sin considerar play-offs ni desempates.

5. Reflexión final y uso de la retroalimentación

El proyecto evolucionó desde una idea inicial centrada en predecir directamente la posición final (variable ordinal con empates) hacia un planteamiento más sólido: predecir `puntos_temporada` y derivar de allí el ranking vía ordenamiento. Este giro, sugerido en la retroalimentación de S4, fue la decisión técnica de mayor impacto en la calidad del prototipo.

Otras retroalimentaciones incorporadas a lo largo del curso:

- S2: organizar los datos por categoría, en vez de un único CSV monolítico, para facilitar la auditoría de fuentes.
- S5: incluir variables rezagadas (lags) de la temporada anterior como proxy de la inercia deportiva.
- S7: comparar al menos tres modelos y reportar métricas de ranking (Spearman) además de las métricas de regresión clásicas.
- S8: priorizar una interfaz limpia y auto explicativa, sustituyendo la primera demo basada en línea de comandos.
- S9: documentar formalmente la robustez del modelo en un anexo independiente y trazable.
- S10: eliminar la dependencia del backend para que el evaluador pueda probar el prototipo abriendo un solo archivo HTML.

Aprendizajes clave del equipo. (i) En datasets pequeños la regularización lineal es difícil de batir; resistir la tentación de escalar a modelos complejos sin justificarlo. (ii) La trazabilidad (`model_card.json`) es tan importante como las métricas mismas: permite que cualquier evaluador reconstruya las decisiones. (iii) La separación en capas (datos / modelo / inferencia) reduce drásticamente el costo de incorporar retroalimentaciones tardías.